Simulation-Optimized Open-Loop Locomotion and an IMU-Adaptive Roadmap for a Twelve-DOF Hexapod

Howard Wang and Matthew Shiley

Engineering 90
Advisor(s): Allan Moser

April 14, 2026

Abstract

This report describes a capstone project on a twelve-degree-of-freedom hexapod actuated through a Pololu Maestro from a Teensy 4.1 microcontroller. The primary completed contribution is a *simulation-to-deployment* pipeline for *open-loop* tripod locomotion: a reduced-order planar body model in Python screens gait parameters with constrained numerical optimization; the best parameter vector exports to firmware that reproduces the same sinusoidal abduction–flexion command law on the robot, with pulse-width clamps and a startup frequency ramp. Quantitative results in this repository are from the surrogate simulation (forward speed, attitude RMS, vertical oscillation, and objective cost). Separately, the repository documents a broader bio-inspired architecture—innate tripod rhythm, fast IMU-driven reflexes, and slow on-device parameter adaptation—with phased exit criteria and timing budgets. That adaptive stack is specified in design memoranda but is not fully implemented in the checked-in firmware; this report therefore emphasizes verifiable open-loop results while treating reflexes and learning as engineering requirements and future work. Benefits include low-dimensional, interpretable parameters, repeatable optimization artifacts, and explicit safety bounds before closing the loop. Project cost to date is tabulated from purchased electronics (Table 2); chassis and remaining servos may increase the final total.

1 Introduction

Legged robots must produce periodic limb motion, reject disturbances, and adjust to terrain under tight sensing and compute budgets. Insects achieve this with layered control: a central rhythm, fast sensory feedback that shapes forces and timing, and slower adaptation of gains. This project applies that philosophy to a tabletop hexapod with hobby servos: the long-term target is IMU-based reflexes and on-device learning on a Teensy 4.1. The *proposed* end state is documented in the repository README and `docs/` tree: tripod gait generation, complementary-filter attitude

estimation, a scalar stability score, reflex modulation of stride and posture, and optional SPSA or bandit updates on a five-dimensional parameter vector with EEPROM persistence.

The *accomplished* milestone captured in version control focuses on a complementary path: **open-loop** sinusoidal gaits optimized offline in simulation and transferred to the `hexapod_openloop_sine` sketch. This choice isolates gait generation and hardware mapping, yields reproducible numerical benchmarks, and establishes a baseline before adding IMU feedback. The benefits are faster iteration in software, clear separation between model error and sensor-based correction, and firmware that mirrors the optimizer's `leg_commands()` function for traceability.

2 Technical Discussion

2.1 Biological and control context

Insect locomotion is often modeled as decentralized feedback atop rhythmic pattern generators [1, 2]. For a single IMU, roll and pitch from accelerometer–gyro fusion are more reliable than yaw when magnetic disturbance is present [3, 4]. Hexapod posture control literature similarly emphasizes body orientation and angular rates on uneven terrain [5]. Disturbance-triggered gait changes are a classical strategy for legged systems [6].

2.2 Layered architecture (specification)

The intended runtime stack, documented in `docs/ARCHITECTURE.md`, has two timescales. A **fast tick** ($\sim$10–20 ms) reads the IMU, fuses attitude, computes a stability score S, applies reflex adjustments to gait variables, steps the gait interpolator, clamps commands, and streams twelve Maestro setpoints. A **slow tick** (each gait cycle) aggregates windowed RMS of S, runs adaptation, and occasionally saves the best parameter vector. The complementary blend

$$\alpha \in (0,1), \quad \theta_{\text{roll}} \leftarrow \alpha(\theta_{\text{roll}} + \omega_x \Delta t) + (1-\alpha)\,\theta_{\text{roll,accel}}, \tag{1}$$

(and analogously for pitch) is the nominal attitude update; Mahony or quaternion filters are noted as upgrades under vibration.

2.3 Open-loop sinusoidal gait (implemented)

For the no-IMU milestone, each leg follows resting abduction and flexion angles plus sinusoidal variation at common frequency f, with tripod group B offset by π radians from group A. The optimizer searches rests, phase shifts, and f; amplitudes are fixed to default tables shared by Python and the generated header `optimized_gait_params.h`. The reduced-order simulation integrates an eight-state planar body with soft contact, damping, and stride thrust proxies; the objective rewards

mean forward speed and penalizes backward motion, attitude oscillation, vertical bounce, torque proxy, contact imbalance, and deviation from a symmetric prior. Optimization uses L-BFGS-B with box constraints and random restarts.

2.4 Hardware and safety (constraints)

`docs/FIRMWARE.md` records the production stack: Teensy 4.1, UART at9600 baud to the Maestro, channels 0–11 in FL/FR/ML/MR/RL/RR order with abduction then flexion per leg, and mapping from abstract joint integers to pulse width $u_k = \text{clamp}(c_k + d_k q_k)$ with typical bounds 600–2400 μs. Open-loop firmware adds degree-to-command gains k_{abd}, k_{flx} and asymmetric command clamps before `jointToUs`; these gains are *not* outputs of the ODE optimizer and must be tuned on the bench.

3 Requirements and Constraints

3.1 Source of requirements

No separate "requirements and constraints memo" file is present in this repository. Table 1 therefore traces engineering requirements to the phase exit criteria in `docs/PHASES.md`, hardware notes in `docs/FIRMWARE.md`, and the open-loop optimization scope in `firmware/hexapod_openloop_sine/README.md`. If your course issued a formal memo, replace or extend this table and paste the memo in an appendix.

3.2 Summary: met vs. not met

Met (repository evidence): discrete tripod baseline (`hexapod_walking_v2`); open-loop optimizer and summary JSON/CSV; firmware `hexapod_openloop_sine` with JSON-to-header script; calibration helpers (`set_zero`, left/right side test sketches); complete design and architecture documentation.

Partial / not in version control: Phase 1 IMU telemetry sketch path listed in README but not present as `hexapod_imu_test/`; Phases 2–4 adaptive firmware paths (`hexapod_adaptive/`) not present; on-device PPO/SPSA described in `docs/on_device_learning/ON_DEVICE_LEARNING_REPORT.md` remains planning.

3.3 Project cost

Table 2 lists electronics purchased for the platform to date (prices as recorded at time of purchase). A full twelve-DOF leg set uses twelve servos; this BOM includes eight DFRobot SER0039 units, so four additional servos (and any frame, fasteners, or printed parts not listed here) should be

Table 1: Requirement traceability (hybrid scope). "Evidence" points to repository artifacts.

ID	Requirement	Status	Evidence / notes
R1	Phase 1: IMU read, complementary filter, USB telemetry at ∼50 Hz; gait unchanged	Not met (in repo)	Specified in `PHASES.md`; folder not in tree
R2	Phase 2: postural reflexes, clamps, serial tuning, EEPROM for gains	Not met (in repo)	`ARCHITECTURE.md` pseudocode only
R3	Phase 3: stability score, cautious mode, push recovery, fall freeze	Not met (in repo)	Documented only
R4	Phase 4: five-parameter adaptation, cost J, SPSA/bandit, EEPROM best-J	Not met (in repo)	Documented only
R5	Hardware: Teensy + Maestro + twelve servos + documented channel map	Met	`FIRMWARE.md`, sketches
R6	Safety: pulse clamps; parameter slew (for adaptive layer)	Partial	Clamps in v2/openloop; slew spec in `ARCHITECTURE.md`
R7	Open-loop: optimizer/firmware parity for sinusoidal law; export JSON	Met	`hexapod_no_imu_optimization.py`, `sync_gait_params_from_json.py`, `optimized_gait_params.h`

budgeted to complete the hexapod. No dollar figures appear in the optimization JSON—only physical parameters.

Vendor product pages are archived in the team purchase records (DigiKey, DFRobot, Amazon).

4 Methods

4.1 Baseline discrete gait

`firmware/hexapod_walking_v2/hexapod_walking_v2.ino` implements a deterministic tripod state machine with smoothed keyframes (`moveSmoothTo`). It is the behavioral reference for "brain v0" in `hexapod_brain_roadmap copy/ROADMAP.md`.

4.2 Simulation optimization

Script `hexapod_brain_roadmap copy/ode_optimization_no_imu/hexapod_no_imu_optimization.py` integrates the reduced-order model with `scipy.integrate.solve_ivp`, evaluates the weighted objective on a steady-state window, and calls `scipy.optimize.minimize` (L-BFGS-B). Outputs include `no_imu_optimization_summary.json`, time series CSV, and optional plots.

Table 2: Bill of materials (electronics purchased to date; USD).

Part	Description	Vendor	Qty	Unit	Line
SparkFun 16771	Microcontroller	DigiKey	1	37.20	37.20
Pololu 1354	18-channel servo driver	DigiKey	1	46.95	46.95
DFRobot DFR1202	Step-down buck converter	DFRobot	1	12.90	12.90
SparkFun BMI270	6-DoF IMU	DigiKey	1	18.50	18.50
SparkFun 11855	2×7.4 V LiPo + charger (URGENEX kit, Amazon listing)	Amazon	1	27.99	27.99
SparkFun 14417	Female Qwiic connector	DigiKey	2	0.60	1.20
SparkFun 17259	100 mm Qwiic cable	DigiKey	1	1.95	1.95
SparkFun 17261	Qwiic cable to female header	DigiKey	1	1.95	1.95
Amphenol Anytek YO0201500000G	1×2 screw-down terminal	DigiKey	2	1.28	2.56
DFRobot SER0039	Servo motor	DigiKey	8	5.90	47.20
Total					**198.40**

4.3 Parameter sync and firmware

From `firmware/hexapod_openloop_sine/`, run `python3 sync_gait_params_from_json.py` (optionally with path to a summary JSON) to regenerate `optimized_gait_params.h`. The sketch `hexapod_openloop_sine.ino` evaluates the sinusoidal law at `INTERP_DELAY_MS` (default 10 ms), applies `FREQ_RAMP_MS` startup ramping, maps degrees to integer commands, clamps, and sends Pololu compact-protocol targets on `Serial1`.

4.4 Bench testing and calibration

`firmware/set_zero/` and side-only test sketches support directional calibration. README in `hexapod_openloop_sine` documents lift-phase sign and scale tuning if feet drag or the robot shuffles backward.

5 Results

5.1 Simulation metrics (from committed summary JSON)

Table 3 summarizes `hexapod_brain_roadmap copy/ode_optimization_no_imu/no_imu_optimization_summary`. These numbers characterize the *surrogate model* after optimization; they are *not* automatic guarantees on hardware without validating k_{abd}, k_{flx}, centers, and directions.

Table 3: Optimizer-reported simulation metrics (committed summary file).

Quantity	Value
Gait frequency f	1.80 Hz
Mean forward speed (steady window)	0.234 m/s
Total displacement (rollout)	1.86 m
Roll RMS	0.440 rad
Pitch RMS	~ 0 (numerically near zero in file)
Vertical position RMS	0.086 m
Peak servo usage ratio (proxy)	0.152
Contact balance std. dev.	0.388
Objective cost	−228.38

5.2 Requirements satisfaction (results view)

Open-loop requirements (R5–R7) are supported by code and artifacts. Phase 1–4 firmware requirements (R1–R4) are **not** satisfied in the repository snapshot; documentation and roadmap materials describe intent and acceptance tests for future integration.

5.3 Cost

See Table 2: electronics purchased to date total **US$198.40**. Additional servos and mechanical hardware are not yet fully captured in that sum.

6 Discussion

Context. The simulation results demonstrate that the surrogate dynamics and objective can select a coherent frequency and phase structure with nontrivial forward progress in the model. The hybrid project framing is intentional: shipping documentation for IMU adaptation without claiming completed adaptive firmware avoids overstating experimental validation.

What went well. A single mathematical command law is shared between Python and C++; JSON-to-header automation reduces transcription error; the existing Maestro and v2 infrastructure provides a stable execution layer.

Limitations. Model mismatch, unmodeled compliance, and servo saturation can invalidate predicted speeds; degree-to-pulse scaling is manual; open-loop control does not reject pushes or slopes. Yaw and magnetometer-based heading remain poor choices without EMI control.

Lessons and advice for future E90 students. Freeze a repeatable baseline gait and logging format before adding sensors. Treat simulation as screening, then budget time for bench tuning. Encode safety clamps and fall-freeze logic before high-energy tests. Keep a short written trace from

optimizer output to firmware constants for examiners and your future self.

7 Conclusion

This report presented (i) a completed open-loop optimization and firmware deployment path for a twelve-DOF hexapod, (ii) quantitative simulation benchmarks committed in JSON form, and (iii) a documented roadmap toward IMU reflexes and slow on-device adaptation that is not yet fully realized in the repository firmware. Major takeaways: layered, bio-inspired control is a credible long-term architecture; the open-loop milestone delivers interpretable parameters and a verifiable toolchain; closing the loop will require IMU integration, validation protocols, and honest reporting of which phase exit criteria are met.

Acknowledgements

We thank our advisor, Allan Moser, for guidance on the project. We also thank Swarthmore Engineering and any shop or lab staff who supported fabrication and bench testing. List here any funding sources beyond the Engineering department if applicable.

References

[1] Y. O. Aydin *et al.*, "Mechanosensory control of locomotion in animals and robots: Moving forward," *Biomimetics*, vol. 8, no. 4, p. 211, 2023. PMC10445419.

[2] G. Chen *et al.*, "Adaptive locomotion control of a hexapod robot via bio-inspired learning," *Frontiers in Neurorobotics*, vol. 15, p. 627157, 2021.

[3] R. G. Valenti, I. Dryanovski, and J. Xiao, "Keeping a good attitude: A quaternion-based orientation filter for IMUs and MARGs," *Sensors*, vol. 15, no. 8, pp. 19302–19330, 2015.

[4] R. Zhang *et al.*, "Attitude estimation algorithm of portable mobile robot based on complementary filter," *Micromachines*, vol. 12, no. 11, p. 1373, 2021.

[5] L. Bai *et al.*, "Development and implementation of a new approach for posture control of a hexapod robot to walk in irregular terrains," *Robotica*, vol. 37, no. 12, pp. 2225–2238, 2019.

[6] A. Johnson, K. D. Gregg, and D. E. Koditschek, "Disturbance detection, identification, and recovery by gait transition in legged robots," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 3627–3634, 2010.

A Repository map (selected paths)

- README.md, docs/DESIGN.md, docs/ARCHITECTURE.md, docs/PHASES.md, docs/FIRMWARE.md
- firmware/hexapod_walking_v2/, firmware/hexapod_openloop_sine/
- firmware/hexapod_test_left_side_only/, firmware/hexapod_test_right_side_only/, firmware/set_zero/
- hexapod_brain_roadmap copy/ode_optimization_no_imu/
- docs/on_device_learning/ON_DEVICE_LEARNING_REPORT.md